

ELEC5620 · STAGE 1 · MON 10-12 GROUP 14

AI Trip Planner

A one-person AI travel agency: seven AI roles plan a trip together, one human runs the company.

A · Ziqi He · Orchestrator

B · Tingsong Jin · Itinerary and transport

C · Yi Qiao · Accommodation and budget

D · Jiahang Bian · Destination and dining

E · Weihao Wang · Workspace and memory

Every travel choice constrains the others

Too many sites

Flights, hotels, maps, food blogs and a budget spreadsheet, each on its own.

Chatbots invent facts

A single model writes a nice itinerary with made-up prices and never checks that the parts fit.

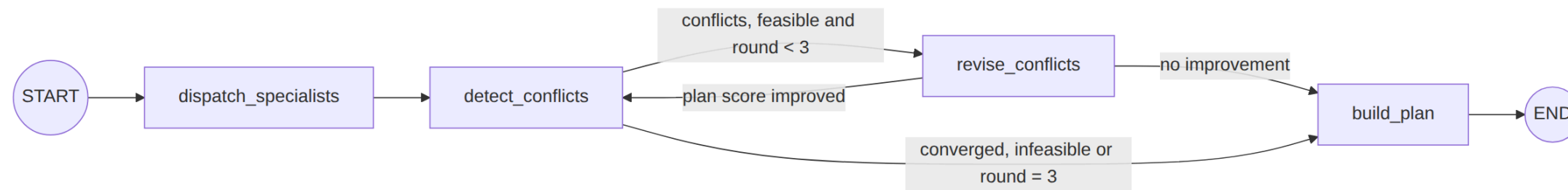
Our answer

A one-person AI travel agency: specialist agents plan together on real data, and code checks their work.

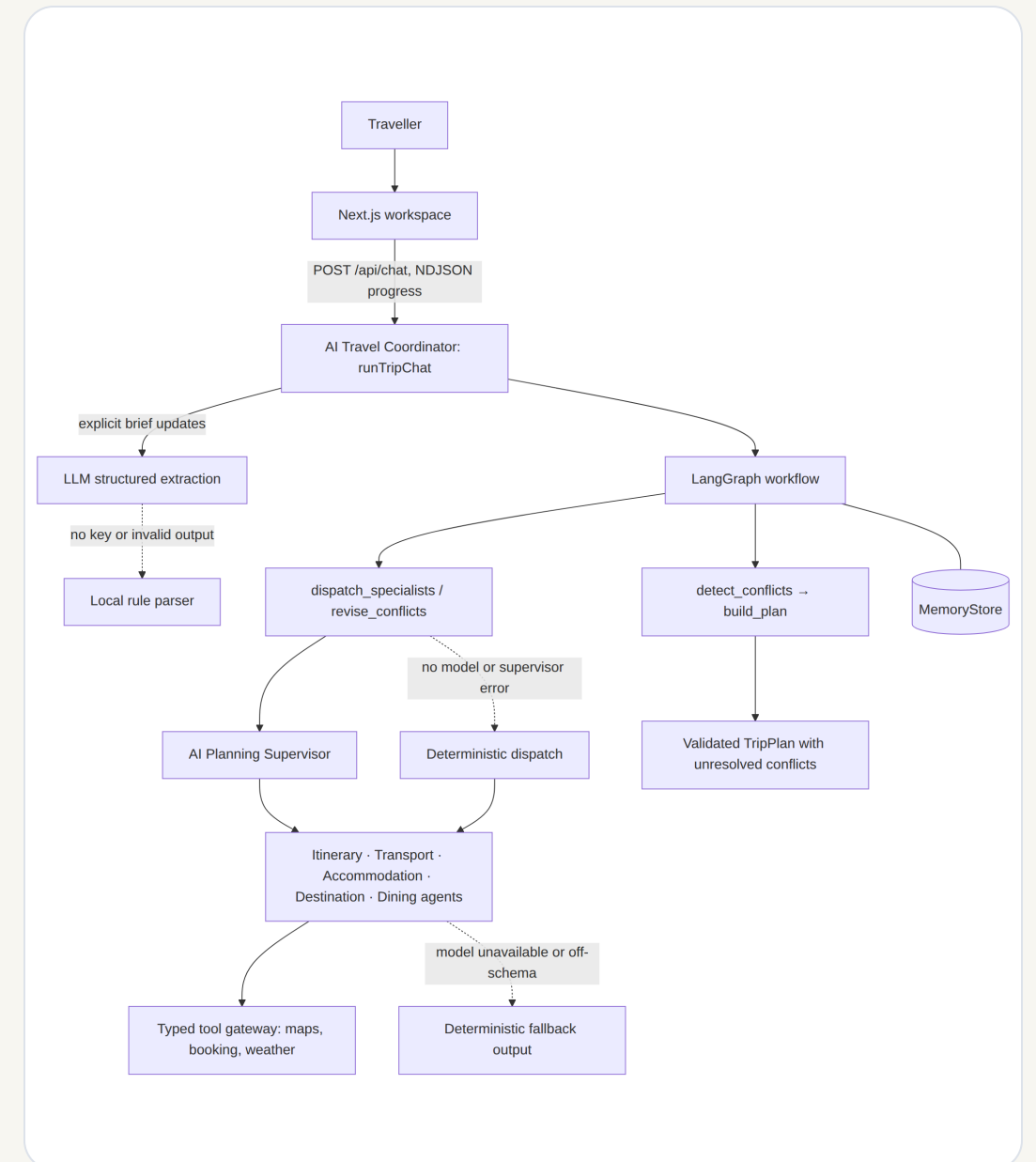
Two humans, seven AI roles

Role	Kind	Responsibility
Traveller	Human user	Describes the trip, reads and edits the plan
Founder / operator	Human	Runs the agency: models, providers, quotas, Agent Lab
Travel Coordinator	AI	Chat, turns it into a TripBrief, asks for missing facts
Planning Supervisor	AI	Picks which specialists each step needs
Itinerary · Transport	AI specialists	Day plan and route checks; flights, rail, local routes
Accommodation · Dining	AI specialists	Stays and room allocation; venues and dietary needs
Destination guide	AI specialist	Attractions, customs, safety, weather and packing

The graph drives the agents, not the other way round



1. Specialists plan in stages on a shared board: transport, then accommodation, then itinerary and dining.
2. Code detects budget, time and route conflicts.
3. Only the agents involved revise, for at most three rounds, and a round is kept only if the plan improves.



[A's use case]: activity, sequence and state machine

[Activity diagram]

[Sequence diagram]

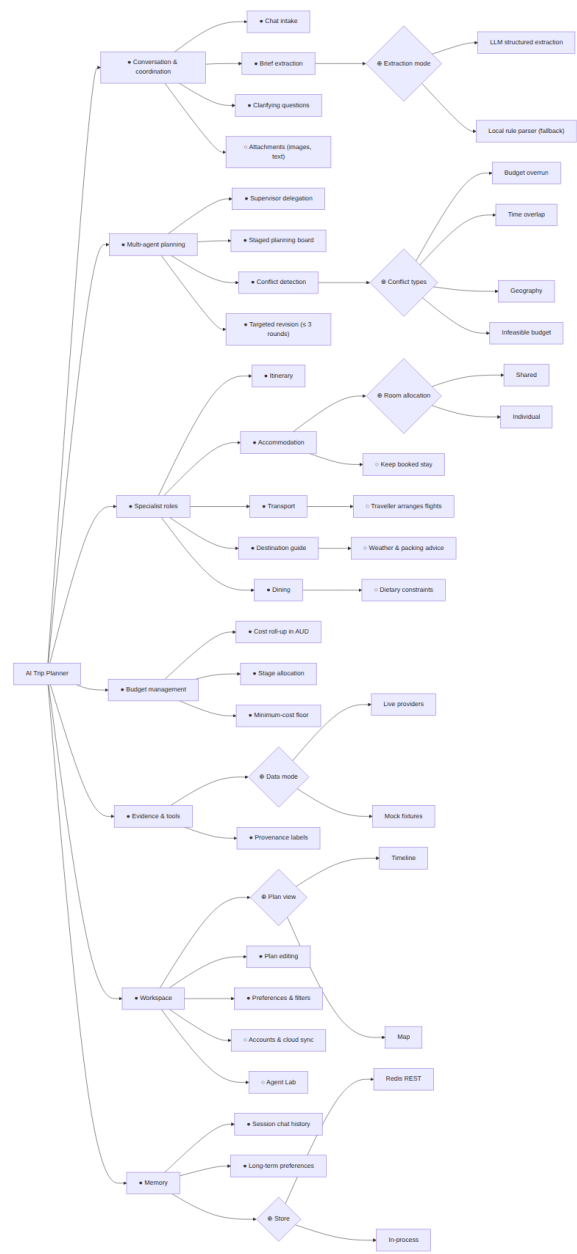
[State machine]

Each member owns two core features and one optional

Who	Core feature 1	Core feature 2	Optional
A	Brief extraction and clarifying questions	Orchestration, conflicts, targeted revision	Agent Lab and Failure Lab
B	Itinerary with route feasibility	Flights, rail and local transport	Traveller-chosen leg mode
C	Accommodation, individual or group	Budget management	Keep a booked stay
D	Destination guide	Dining with dietary needs	Weather-based packing
E	Chat, timeline and map editing	Preferences and memory	Accounts and cloud sync

Agreement: a single-user planner with AUD estimates; no booking or payment. Models choose, code decides: no invented prices, no relaxed preferences.

Eight feature groups with their constraints



Notation

• mandatory · ○ optional · ⊕ exactly one · ⊗ one or more

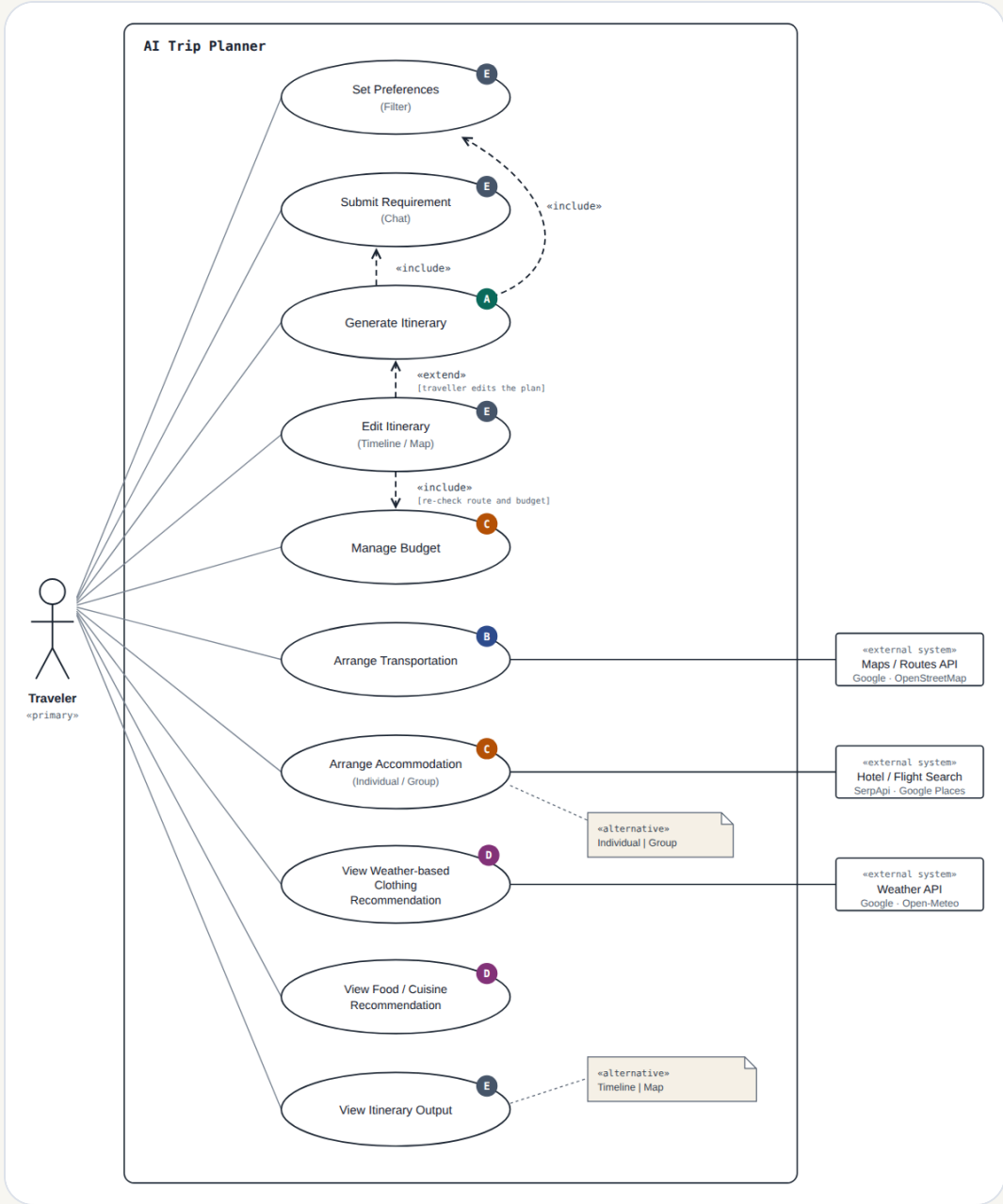
Cross-tree constraints

Targeted revision requires conflict detection. Infeasible budget requires the minimum-cost floor. Live providers require keys, else mock fixtures.

Non-functional

Schema checks at every boundary, money in integer cents, a provenance label on every section.

Ten use cases, one traveller, three external systems



Use case	Owner, relation
Generate Itinerary	A, «include» 3
Arrange Transportation	B, Maps API
Arrange Accommodation	C, Hotel search
Manage Budget	C, «extend» Edit
Weather clothing advice	D, Weather API
Food recommendation	D
Set Preferences, Submit Requirement, Edit, View	E

[B's use case]: activity, sequence and state machine

[Activity diagram]

[Sequence diagram]

[State machine]

From one ad hoc requirement to two use cases

"Flights get paid first, so the hotel has to fit in whatever money is left. If the trip ends up over budget, swap the hotel for a cheaper one that still meets my rules. If no hotel can ever fit, tell me the minimum I'd need."

AH-C1, in the traveller's words

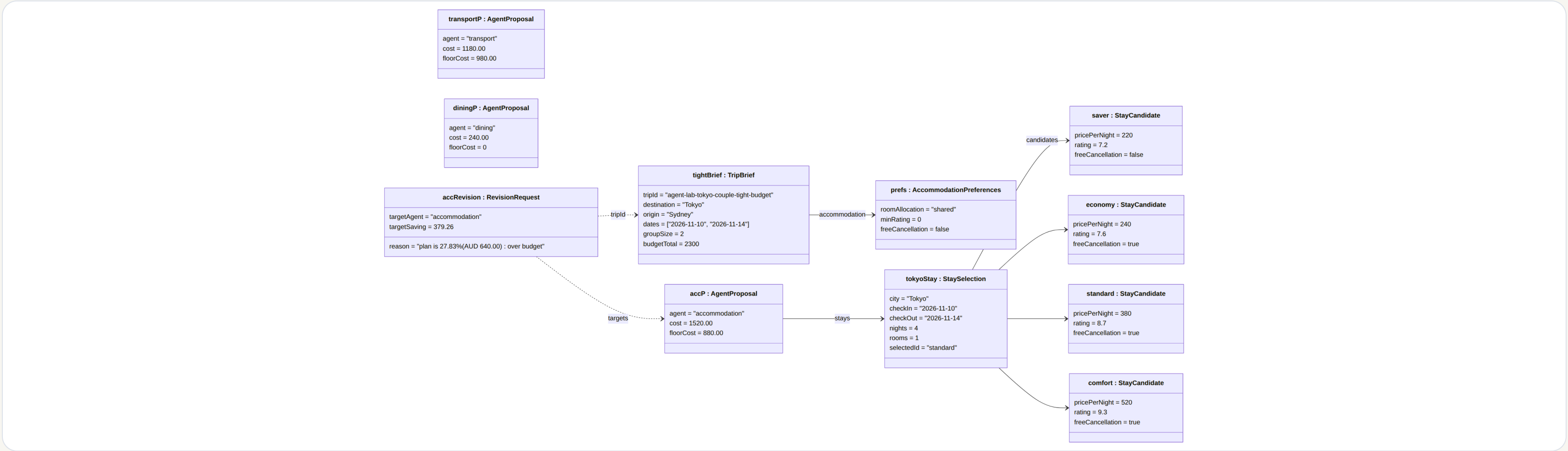
UC-C1 Arrange Accommodation

Rooms from party size, filter by rating and cancellation, the LLM picks one candidate id, code validates and prices it in cents.

UC-C2 Manage Budget

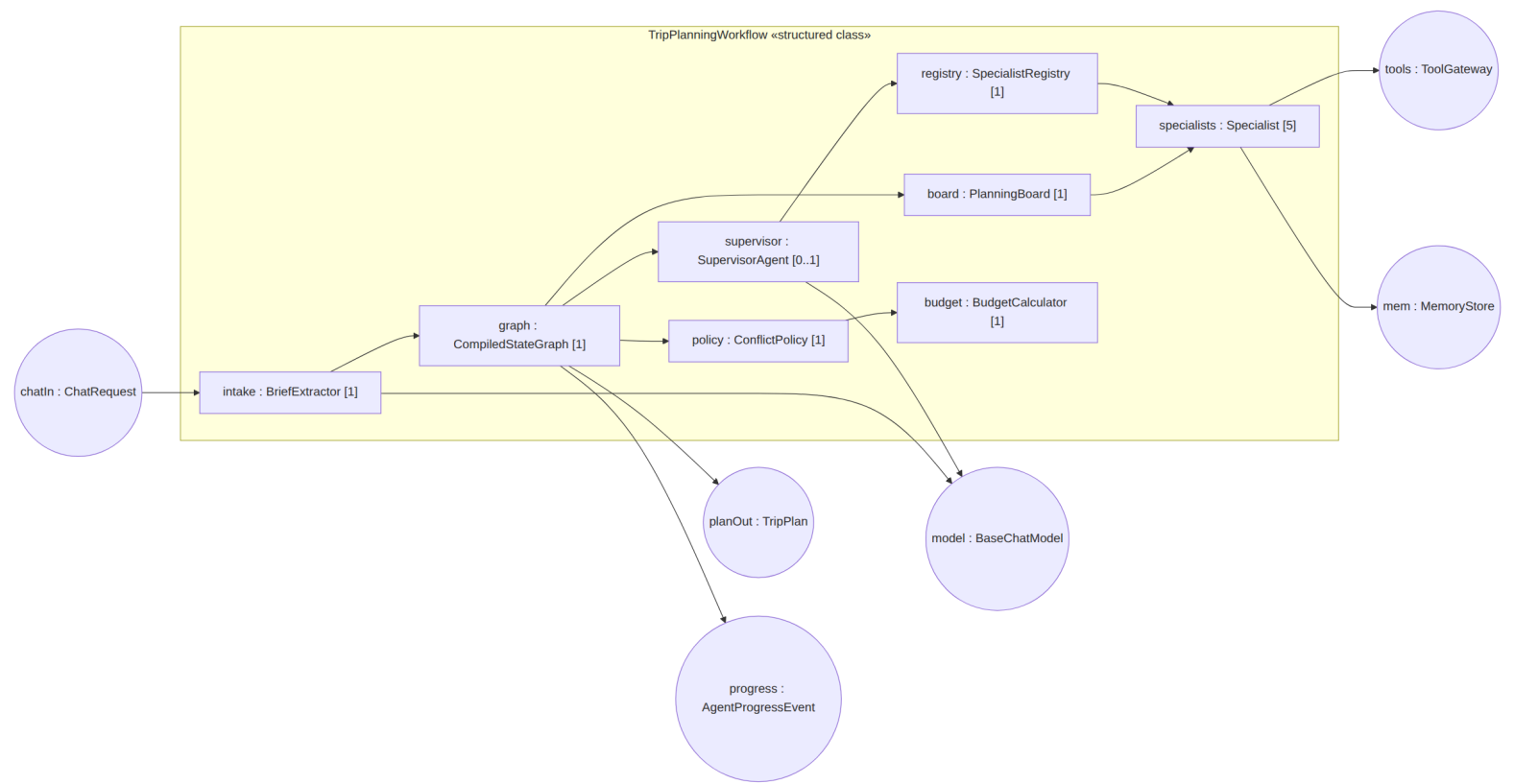
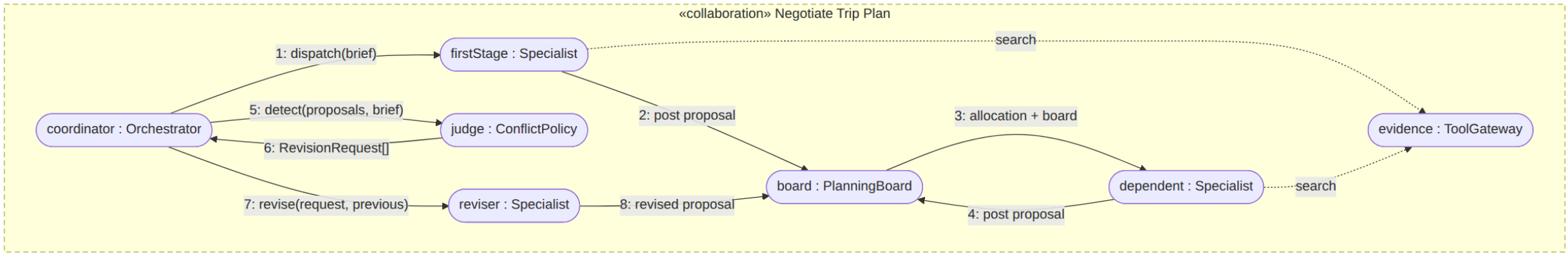
Roll up costs, spread an overrun over what each section can still cut, or report the minimum budget when nothing can fit.

Object diagram: a Tokyo trip that is AUD 640 over budget

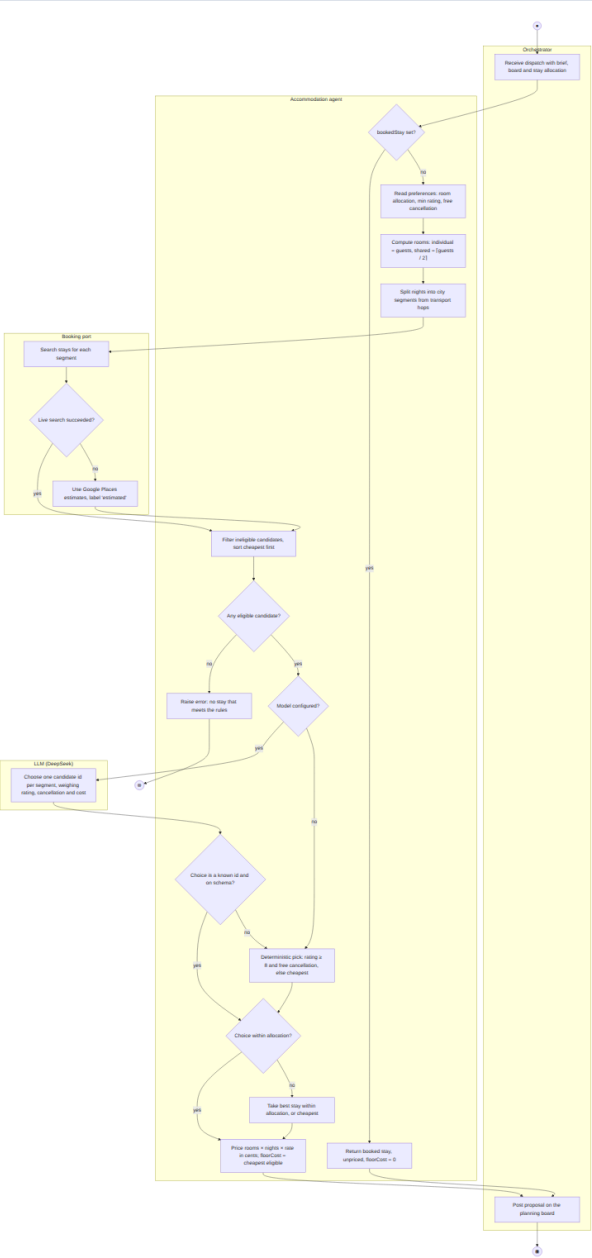


Budget AUD 2,300, total AUD 2,940. The overrun is spread by room to cut, so accommodation is asked to save **AUD 379.26** and moves from *standard* (AUD 1,520) to *saver* (AUD 880).

Collaboration and structured class



The LLM chooses once; code checks before any price



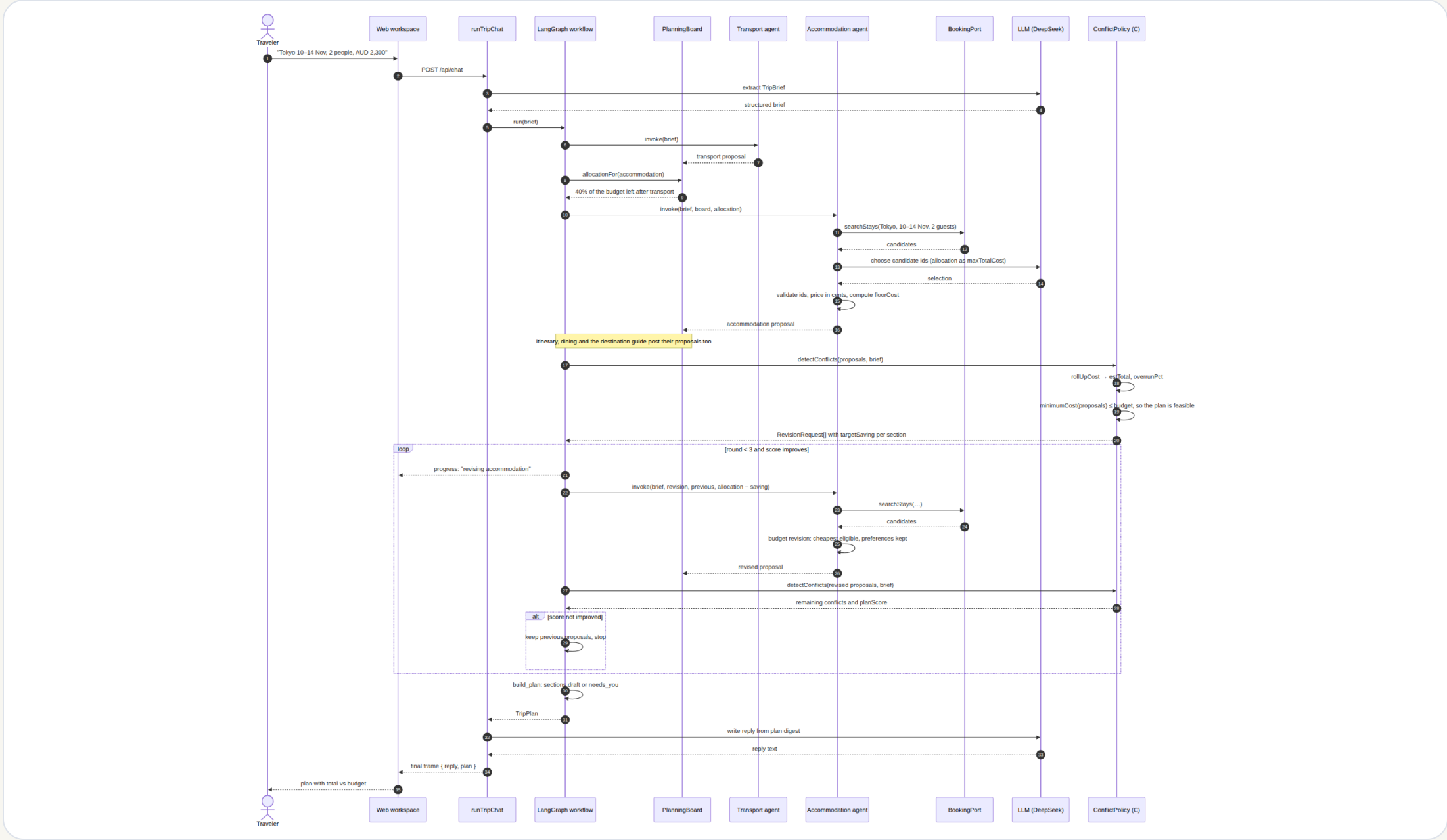
Four swimlanes: orchestrator, accommodation agent, booking port, LLM.

LLM step: choose one candidate id per stay.

Guards: unknown id or off-schema output falls back to a deterministic pick; a booked stay skips the search.

Output: stays priced in cents, plus the cheapest eligible total as the floor.

Budget overrun and a targeted revision



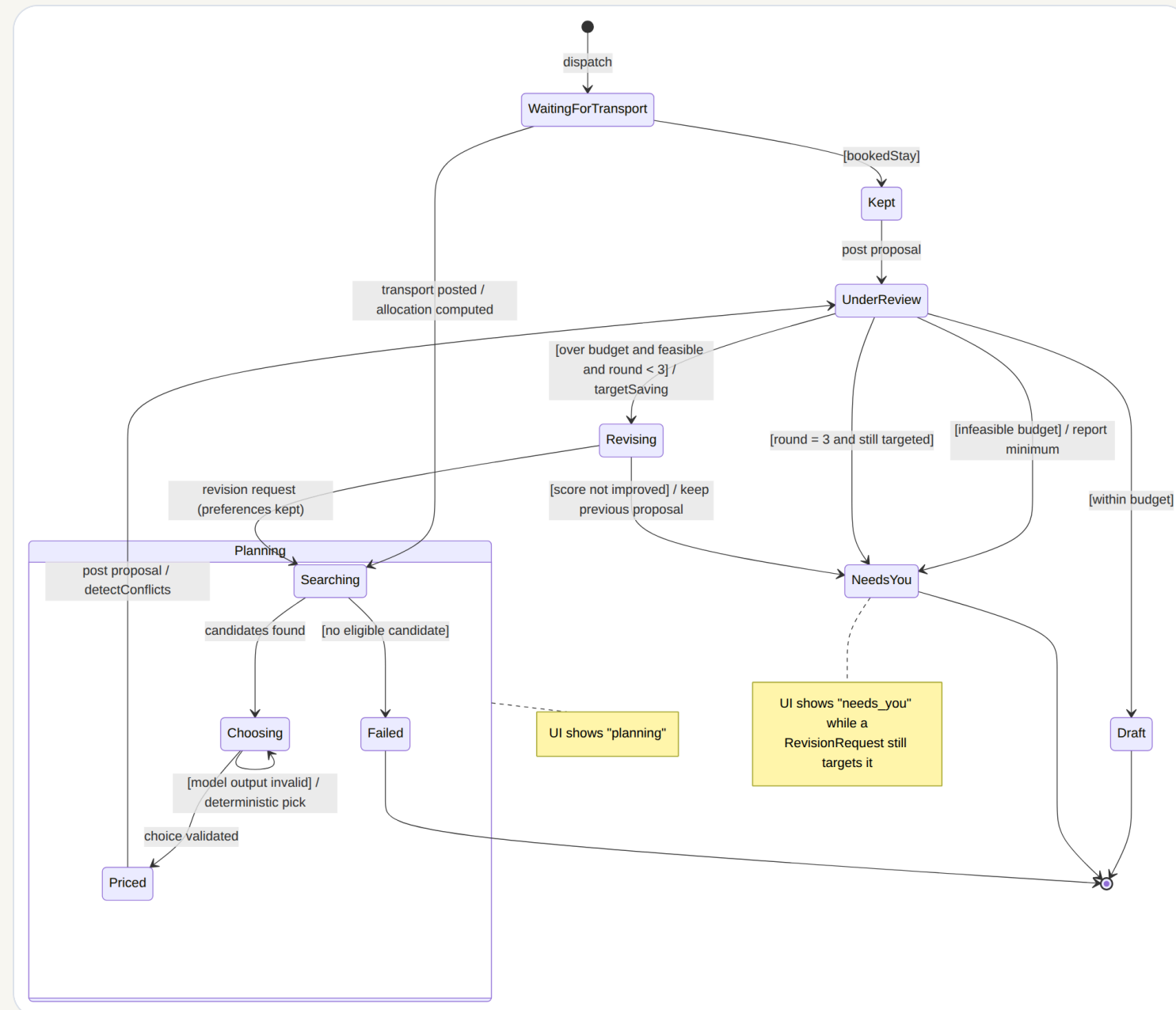
Round 1: total over budget.

detectConflicts: a revision request with a target saving.

Round 2: only the targeted agents revise; kept if the score improves.

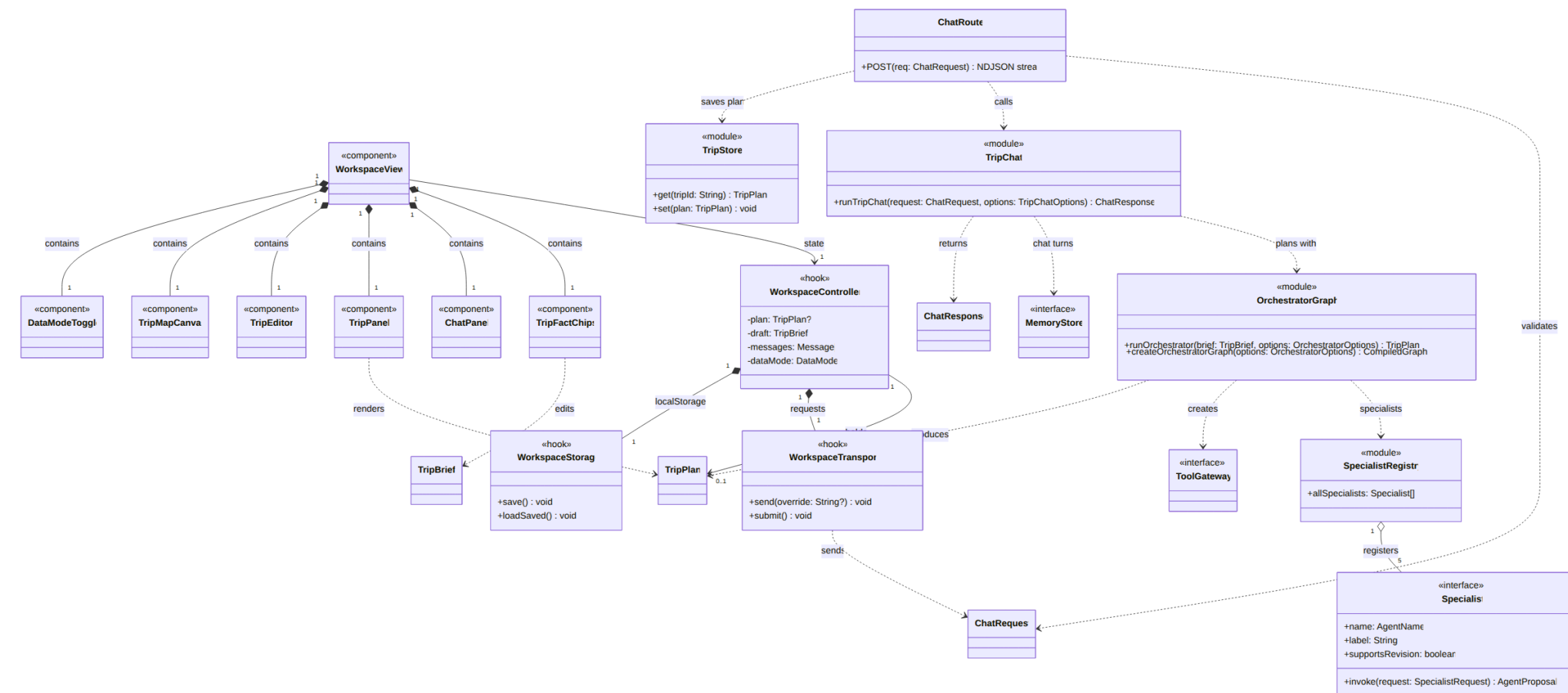
Infeasible: one conflict naming the minimum budget.

The accommodation section within one planning turn



Internal state	Shown as
Waiting, Planning, Kept, UnderReview, Revising	planning
Draft	draft
NeedsYou	needs_you
Failed	run stops

Six class diagrams drawn from the code



1 Structural spine

2 Domain contracts

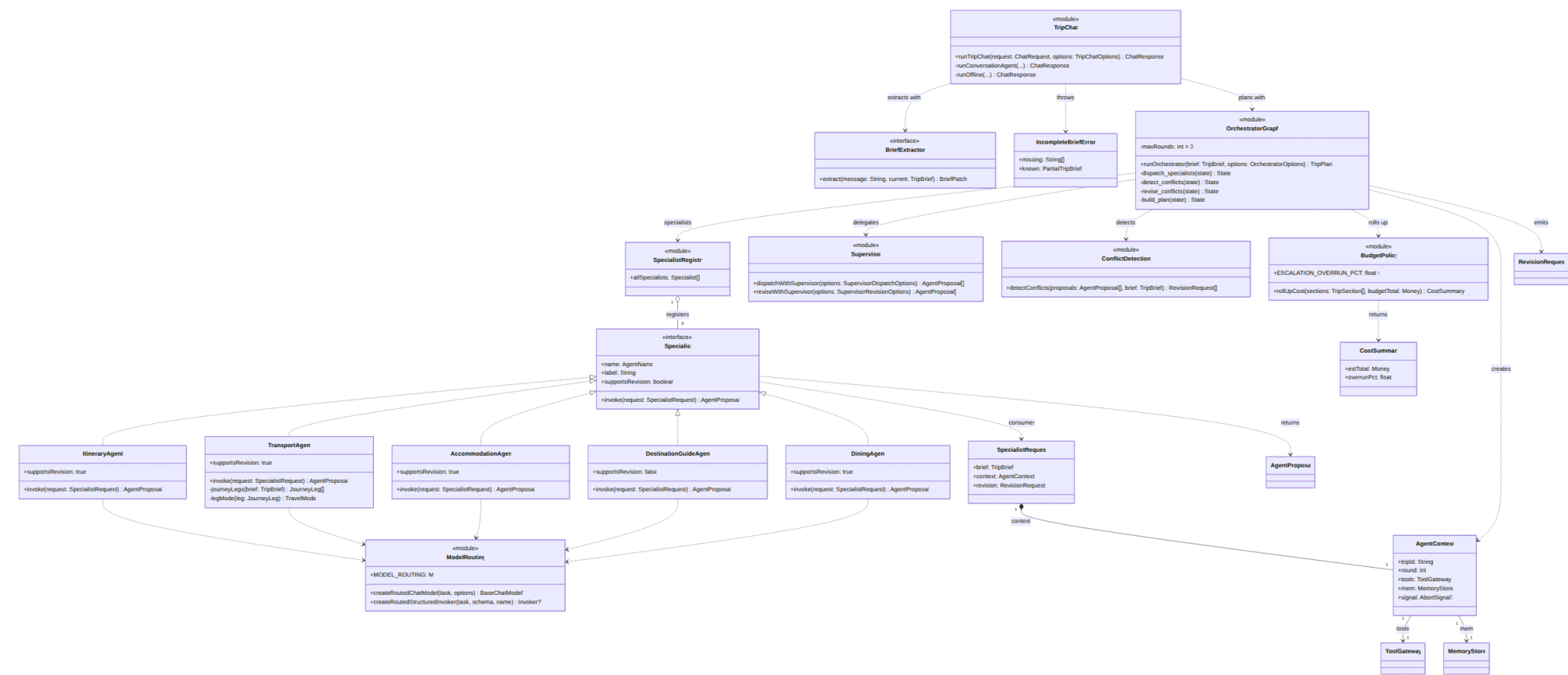
3 Specialists

4 Ports and adapters

5 Use case trace

6 Generalisation

One Specialist interface, five realisations



Relationship	Kind	Why
Registry ◇ Specialist	aggregation 1 → 5	agents live on their own; the registry only lists them
TripPlan ◆ TripSection	composition 1 → 0..*	a section cannot outlive its plan

What we chose, and what we discarded

Decision	Chosen	Discarded, and why
Who drives the loop	Deterministic LangGraph	A model-driven loop: limits and costs must be testable
Collaboration	Staged board with budget shares	Free agent-to-agent chat: control flow in model output
Human in the loop	Traveller edits; sections show needs_you	Approval checkpoints: confirming applied nothing
LLM provider	DeepSeek	MiniMax: pages took 37–81 s against 15–23 s
Prices	Tools only, with provenance	Model estimates: it invents hotels and fares
Server state	Redis REST store	Process memory: lost on cold starts

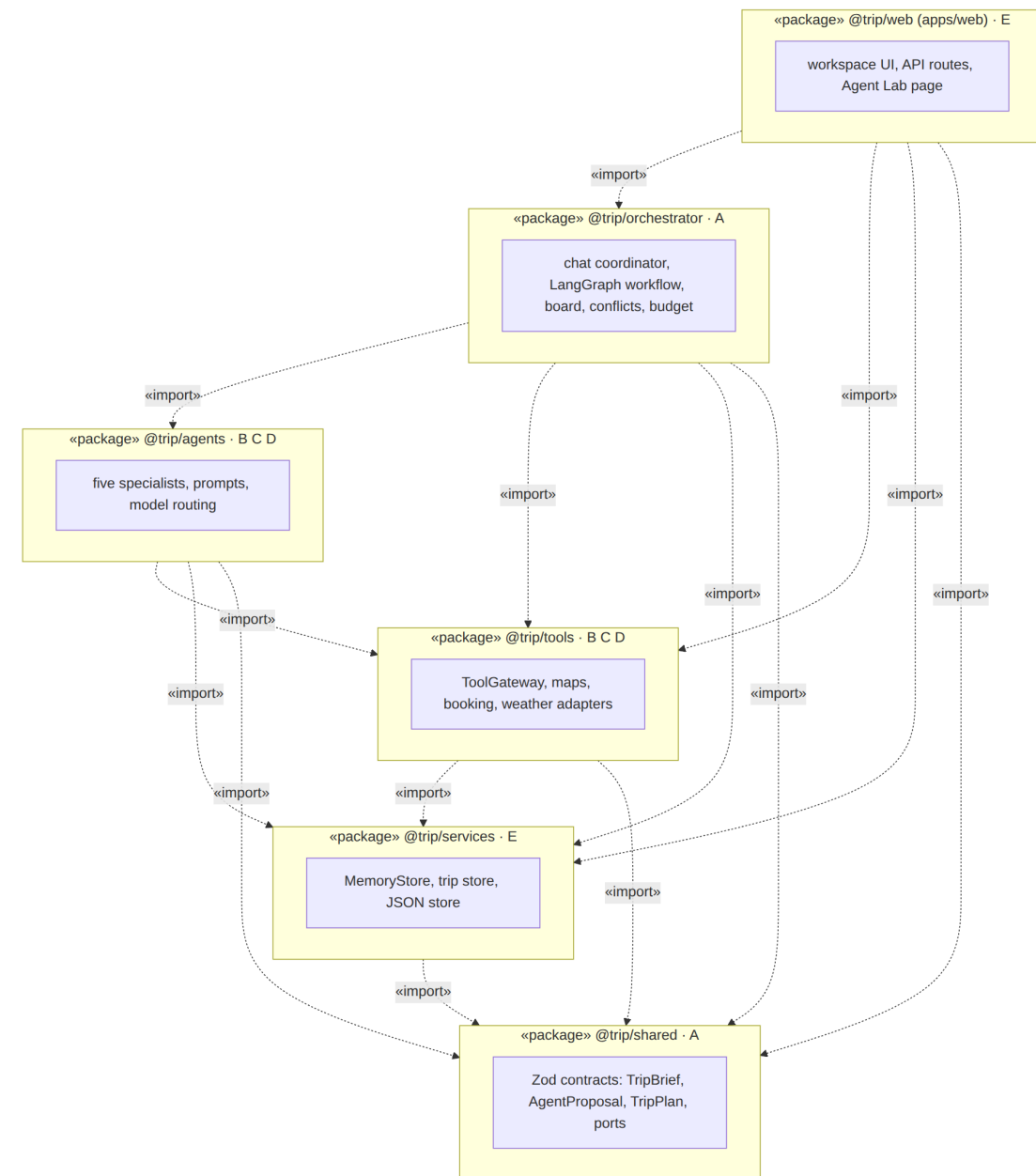
[D's use case]: activity, sequence and state machine

[Activity diagram]

[Sequence diagram]

[State machine]

Dependencies point inward to the shared contracts



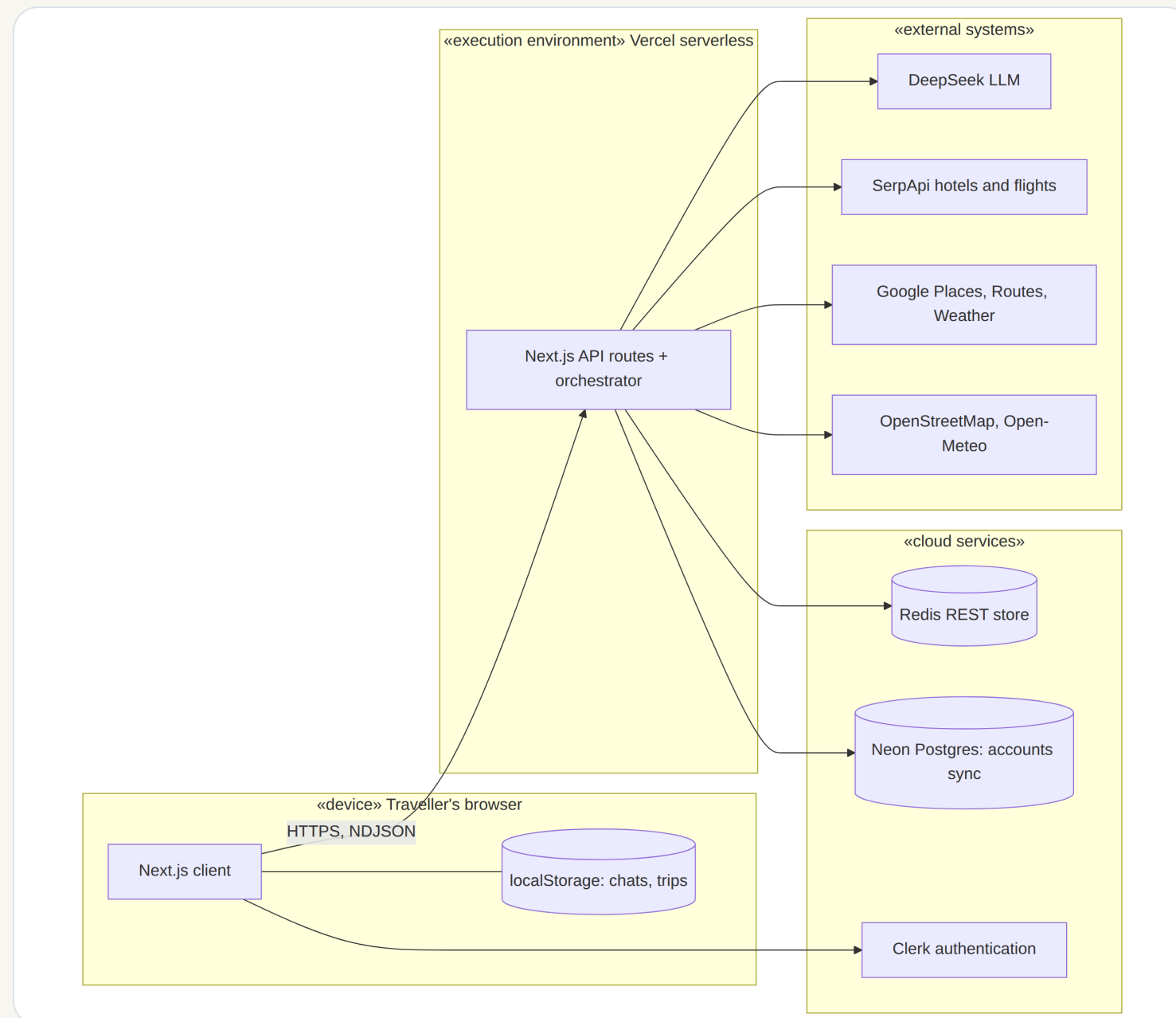
Six packages in one monorepo, each with an owner.

shared holds the Zod contracts and ports and depends on nothing.

No package imports the web app, so the planner runs and is tested without a browser.

Future work plugs in at interfaces: a new specialist, provider or store.

One serverless app, optional cloud services



Browser: chats and trips in local storage; progress streams as NDJSON.

Server: Next.js API routes and the orchestrator.

Optional: Redis REST memory, Postgres account sync, Clerk sign-in.

No keys? The same build runs offline on mock data.

How a change gets in

Assess

- Trace it to a requirement and use case
- Shared contract touched? Write a decision note and tell every owner
- Reversing a decision? Supersede it in writing

Accept

- CI green: types, lint, tests, build, docs
- End-to-end test leaves a repeatable artifact
- Use case postconditions still hold
- Docs and models updated in the same pull request

Key assumptions: one traveller per trip, AUD everywhere, estimates not bookings, flights paid first, models and providers can fail at any time.

[E's use case]: activity, sequence and state machine

[Activity diagram]

[Sequence diagram]

[State machine]

SUMMARY

Models choose. Code decides. The traveller stays in control.

Stage 2: verify live providers end to end, extend the Agent Lab, and track the work in Jira.

github.com/Lilstanie/AI_TRIP_PLANNER